Killer Sudoku

Solver, Generator and Difficulty Rating

Developer Documentation

David Sedláček

July 2026

Contents

1 Problem Description

This project implements a set of tools for working with Killer Sudoku in Prolog — a variant of the classic Sudoku puzzle extended with additional constraints in the form of *cages*.

The project consists of three main components:

1. **Solver** — accepts a Killer Sudoku puzzle and finds a valid solution, or reports that none exists.
2. **Generator** — creates a new Killer Sudoku puzzle for a given grid size (4×4, 6×6, 9×9, 12×12), with a best-effort uniqueness guarantee.
3. **Difficulty rating** — estimates the difficulty of a given Killer Sudoku puzzle (`easy` / `medium` / `hard`).

2 Formal Problem Definition

2.1 Grid

A grid of size $N \in \{4, 6, 9, 12\}$ is a matrix $G = (g_{ij})$ where $i, j \in \{1, \ldots, N\}$ and $g_{ij} \in \{1, \ldots, N\}$.

Block sizes for each grid:

Grid size	Block size
4×4	2×2
6×6	2×3
9×9	3×3
12×12	3×4

2.2 Classic Sudoku Rules

- **Rows**: each row contains every number 1–N exactly once.
- **Columns**: each column contains every number 1–N exactly once.
- **Blocks**: each block contains every number 1–N exactly once.

2.3 Killer Sudoku Extension

The grid is divided into disjoint groups of cells $C_1, \ldots, C_k$ (called *cages*) that together cover the entire grid. Each cage C_i is assigned a target sum s_i. The following must hold:

- The sum of values in each cage equals its target: $\sum_{(r,c)\in C_i} g_{rc} = s_i$.
- No two cells in the same cage may have the same value.

3 Algorithm Design

3.1 Solver

The solver uses **Constraint Logic Programming over Finite Domains** (`library(clpfd)`) in SWI-Prolog.

1. Create CLP(FD) variables for all cells with domain $1..N$.
2. Add constraints for rows, columns and blocks (`all_distinct/1`).
3. Add cage constraints: sum (`sum/3` with `#=`) and uniqueness (`all_distinct/1`).
4. Run labeling with the `[ffc]` heuristic.

The `ffc` heuristic selects the variable with the smallest domain that participates in the most constraints. This prunes the search tree early and is well suited for Killer Sudoku where cage constraints significantly reduce domains.

3.2 Generator

1. Generate a random valid grid using CLP(FD) labeling with a randomly shuffled variable order (`random_permutation/2`).
2. Partition the grid into valid connected cages of at most `MaxSize` cells.
3. Compute the target sum for each cage from the generated grid.
4. Verify uniqueness using `is_unique/3` with a 15-second timeout. If the timeout expires, the puzzle is returned without a uniqueness guarantee.

The maximum cage size is determined by difficulty:

Difficulty	Max cage size
easy	2
medium	3
hard	4

3.3 Difficulty Rating

Difficulty is estimated heuristically from three factors combined into a weighted score $s \in [0, 1]$:

$$s = 0.4 \cdot F_1 + 0.3 \cdot F_2 + 0.3 \cdot F_3$$

- F_1 — average cage size divided by N (maximum possible cage size). Larger cages provide fewer constraints, making the puzzle harder.
- F_2 — number of cages divided by N^2, subtracted from 1. Fewer cages means less information for the solver.
- F_3 — number of solver inferences measured during solving, normalized logarithmically using empirically determined baseline and range values per grid size. More inferences indicate a harder puzzle.

The score is mapped to a difficulty category:

Score range	Difficulty
$s \leq 0.35$	easy
$s \leq 0.60$	medium
$s > 0.60$	hard

4 Code Structure

The project is organised into six modules:

File	Responsibility
`killer_sudoku.pl`	Entry point, welcome message
`main.pl`	Public interface (`play/2`)
`solver.pl`	CLP(FD) solver, validity checking
`generator.pl`	Puzzle generation
`difficulty.pl`	Difficulty rating
`display.pl`	Console output
`utils.pl`	Shared grid utilities

4.1 Module Dependencies

```
killer_sudoku.pl
    +-- main.pl
            +-- solver.pl
            |       +-- utils.pl
            +-- generator.pl
            |       +-- solver.pl
            |       +-- utils.pl
            +-- difficulty.pl
            |       +-- solver.pl
            |       +-- utils.pl
            +-- display.pl
                    +-- utils.pl
```

4.2 Key Predicates

4.2.1 utils.pl

- `get_columns(+Grid, -Columns)` — transposes the grid by extracting each column using `nth1/3` with `maplist/3`.
- `get_blocks(+Grid, -Blocks)` — splits the grid into sub-blocks by first chunking rows into groups of `BR`, then extracting column blocks from each group.
- `block_size(+N, -BR, -BC)` — maps grid size to block dimensions.
- `cell_value(+Grid, +Row, +Col, -Value)` — retrieves a cell value at a given position using `nth1/3`.

- `grid_size_from_cages(+Cages, -N)` — derives grid size by summing all cage cell counts and taking the square root, since cages cover the entire grid ($\sum |C_i| = N^2$).

4.2.2 solver.pl

- `solve_puzzle(+Cages, -Solution)` — public solver interface. Infers grid size from cages using `grid_size_from_cages/2`, then calls `solve_helper/3`. Returns `no_solution` if no solution exists.
- `solve_helper(+N, +Cages, -Solution)` — core CLP(FD) solving predicate. Creates an $N \times N$ grid of variables with domain $1..N$, applies all constraints and runs `labeling([ffc])`. Fails if no solution exists.
- `apply_cage(+Grid, +Cage)` — applies CLP(FD) constraints for one cage: `sum(Cells, #=, Sum)` for the target sum and `all_distinct/1` for cell uniqueness.
- `is_valid(+N, +Grid, +Cages)` — verifies a filled grid against all Sudoku and cage constraints. Used during uniqueness checking in the generator.
- `check_cage(+Grid, +Cage)` — verifies that cage cells sum to the target and contains no duplicate values.
- `find_all_solutions(+N, +Cages, -Solutions)` — collects all solutions using `findall/3`. Used for uniqueness verification.

4.2.3 generator.pl

- `generate(+N, +Difficulty, -Cages)` — main generation predicate. Wraps `generate/4` and discards the solution grid.
- `generate(+N, +Difficulty, -Cages, -Grid)` — generates a puzzle and returns the solution grid. Uses a 15-second alarm to bound uniqueness verification.
- `generate_random_grid(+N, -Grid)` — generates a random valid Sudoku grid using CLP(FD) with `random_permutation/2` to shuffle variable order before labeling.
- `generate_cages(+N, +Grid, +MaxSize, -Cages)` — partitions all grid cells into connected cages by randomly shuffling coordinates and calling `assign_cages/4`.
- `assign_cage(+Last, +Current, +Available, +MaxSize, +Grid, -Cage, -Remaining)` — grows one cage by repeatedly picking a random valid adjacent cell. A cell is valid if its value does not duplicate any value already in the cage (`no_duplicate/3`).
- `compute_cage_sums(+Grid, +RawCages, -Cages)` — computes the target sum for each cage by summing cell values from the solution grid.
- `is_unique(+N, +Grid, +Cages)` — verifies that `Grid` is the only valid solution by checking `is_valid/3` and confirming the solver finds no other solution.

4.2.4 difficulty.pl

- `grade(+Cages, -Difficulty)` — estimates puzzle difficulty. Measures solver inferences using `statistics/2` before and after calling `solve_helper/3`, then combines all three factors into a score via `compute_score/5`.

- `compute_score(+AvgSize, +NumCages, +N, +Inferences, -Score)` — computes difficulty score in $[0, 1]$ from the three factors F_1, F_2, F_3.
- `baseline_log(+N, -Baseline, -Range)` — empirically determined log-inference interval per grid size. Easy puzzles score near `Baseline`, hard puzzles near `Baseline + Range`.
- `score_to_difficulty(+Score, -Difficulty)` — maps the numeric score to `easy` (≤ 0.35), `medium` (≤ 0.60), or `hard`.

4.2.5 main.pl

- `play(+N, +Difficulty)` — generates, displays and solves a puzzle. Calls `generate/3`, `display_puzzle/1`, `solve_puzzle/2`, `display_grid/1` and `grade/2` in sequence. Prints both the requested and estimated difficulty at the end.

5 Known Limitations

- **Uniqueness timeout**: uniqueness verification is bounded by a 15-second alarm (adjustable in `generate/4`). For 12×12 grids or hard difficulty, the puzzle may be returned without a uniqueness guarantee.
- **Solver performance**: solving 12×12 hard difficulty puzzles can be slow due to the size of the CLP(FD) search space.
- **Difficulty rating accuracy**: the heuristic rating is an estimate based on empirically tuned constants. It may not always match the requested difficulty level.